

Curso Inicial Front-End

Clase 07: CSS Flexbox

Módulo 1: Arquitectura y Maquetación Moderna (HTML y CSS)



Presentación

Olvídate de los problemas para centrar un elemento. En esta clase conoceremos **Flexbox**, el modelo de diseño unidimensional que revolucionó la maquetación web. Aprenderemos a distribuir el espacio y a alinear elementos de forma flexible, dinámica y profesional, eliminando para siempre los hacks del pasado.



Objetivos

- Dominar las propiedades del Contenedor (Flex-direction, Justify-content, Align-items).
- Manipular el comportamiento de los hijos (Flex-grow, Flex-shrink, Flex-basis).
- Implementar espaciados modernos usando **gap** y entender su diferencia técnica con el margen.



Bloques Temáticos

- Tema 1: Alineación y distribución dinámica del espacio.
 - El Concepto de Padre e Hijo: El Autobús y los Pasajeros
 - Los Ejes de **Flexbox**: El Puntero Láser
 - Propiedades del Contenedor (Órdenes del Autobús)
 - El Espaciado Moderno: **gap** (La bendición de **CSS**)
- Tema 2: Contenedores e ítems modernos sin dependencia de grillas complejas.
 - Personalidad de los Pasajeros (Items Hijos)

En la clase anterior descubrimos cómo usar `position: absolute` para forzar a una caja a ir a una esquina. Sin embargo, usar posicionamiento absoluto para construir toda la cuadrícula o layout general de tu página (como alinear 5 tarjetas de productos en el centro de la pantalla) es una auténtica pesadilla matemática.

Para salvarnos de hacer cálculos manuales, nació **Flexbox (Flexible Box Layout)**. Es un sistema automático y moderno de diseño en **CSS**. Se inventó exclusivamente para alinear, centrar y distribuir el espacio entre cajas de manera perfecta, ¡incluso si no sabes el tamaño exacto de lo que estás alineando!

 **Recurso Oficial:** [MDN Web Docs - Flexbox](#) | [CSS-Tricks: Guía Visual Definitiva de Flexbox](#)

Tema 1: Alineación y distribución dinámica del espacio.

| Flexbox: Pros y Contras

✓ Ventajas (Pros)	✗ Desventajas (Contras)
Alineación Perfecta al instante: Centrar un elemento hija vertical y horizontalmente dentro de su padre pasó de ser un trauma a costar solo 3 líneas de código.	Diseño Unidimensional: Se empieza a complicar drásticamente si intentas crear layouts de cuadrícula de tablero de ajedrez gigante.
Espaciado Inteligente Automático: Propiedades mágicas como <code>gap</code> hacen el trabajo matemático de calcular los espacios sobrantes por ti.	Curva de Orientación Inicial: Entender cuándo las propiedades se giran de vertical a horizontal suele confundir al cerebro al principio.

| Comparativa: Flexbox vs CSS Grid

Muchos estudiantes dudan sobre cuál de las dos tecnologías aprender o usar. La respuesta profesional es simple: ¡Se usan en conjunto, no compiten!

Característica	Flexbox (<code>display: flex</code>)	CSS Grid (<code>display: grid</code>)
Dimensiones que controla	Unidimensional: Se encarga de controlar 1 solo eje a la vez (alinear en Fila O en Columna).	Bidimensional: Controla la gran cuadrícula entera (Filas Y Columnas al mismo tiempo).
¿Quién manda? (Enfoque)	Guiado por el Contenido (Las cajitas hijas empujan al padre desde adentro hacia afuera definiendo su propio tamaño).	Guiado por el Contenedor (El Padre dibuja un "tablero" vacío estricto y asigna a las hijas en las celdas).
Uso Ideal en el trabajo	Menús de navegación, listas de botones horizontales, centrar un icono junto al texto. Cuidar los micro-detalles.	Maquetar el esqueleto enorme de la página entera (El Header arriba, Sidebar a la izquierda, Footer abajo).

1. El Concepto de Padre e Hijo: El Autobús y los Pasajeros

Flexbox funciona estrictamente mediante una relación de dictadura y obediencia entre dos tipos de cajas:

1. **Flex Container (El Autobús Padre):** Es el gran contenedor al que le inyectamos la propiedad mágica `display: flex`. A partir de ese momento, él toma el control absoluto y pone las reglas: *"¡Atención hijos! Todos deben alinearse mirando al frente, separados por 10 centímetros"*.
2. **Flex Items (Los Pasajeros Hijos):** Son los elementos directos (primer nivel de profundidad) que viven dentro del contenedor. Ellos se transforman automáticamente en pasajeros obedientes, aunque tienen cierto permiso para pelear entre ellos por el espacio sobrante de los asientos.

Traducción Técnica: Al activar `display: flex`, el navegador cambia el algoritmo de renderizado de sus hijos de **Block/Inline Flow** a **Flex Layout**. Esto significa que propiedades como `float` o `vertical-align` dejan de tener efecto en los hijos, ya que el padre toma el control total del cálculo de las coordenadas X e Y de cada caja.

Regla estricta: Al activar `display: flex` en el padre, este NO afecta a los "nietos". Solo ordena a sus hijos directos.

El Espaciado Moderno: `gap` vs `margin`

Antiguamente, para separar elementos en **Flexbox** usábamos `margin`. Pero eso causaba problemas (el primer y último elemento también tenían margen). La propiedad `gap` soluciona esto: - `gap`: Crea un espacio **solo entre** los elementos hijos. No agrega espacio por fuera del contenedor. - `row-gap`: Define el espacio solo entre las filas (si hay `flex-wrap`). - `column-gap`: Define el espacio solo entre las columnas.



TIP

Diferencia clave: El `margin` pertenece al hijo (empuja hacia afuera). El `gap` pertenece al padre (gestiona el vacío entre sus hijos). ¡Usa siempre `gap` en **Flexbox**!

2. Propiedades del Contenedor (Órdenes del Autobús)

Alineación en el Eje Principal (`justify-content`)

Controla cómo se reparte el espacio blanco sobrante a lo largo de tu línea:

- **flex-start**: Empuja a todos al principio (por defecto).
- **center**: Centra a todas las cajas en bloque al medio.
- **flex-end**: Empuja a todos al final del contenedor.
- **space-between**: Manda a los extremos y reparte el espacio en medio (Ideal para **Logo | ---- | Menú**).
- **space-around**: Reparte el espacio de forma pareja alrededor de cada caja.
- **space-evenly**: Reparte el espacio de forma matemáticamente idéntica en todos los huecos.

Alineación en el Eje Secundario (**align-items**)

Controla cómo se comportan las hijas cruzando el láser (alineación perpendicular):

- **stretch** (por defecto): Estira las hijas de arriba hacia abajo para llenar el vacío del autobús.
- **center**: Las centra en el cruce exacto (vital para centrar textos junto a iconos de diferentes alturas).
- **flex-start** / **flex-end**: Las alinea todas pegadas arriba o pegadas abajo.

Comportamiento de Envoltura (**flex-wrap**)

- **nowrap** (por defecto): El autobús es autoritario. Obliga a todos a estar en una fila aplastándose. Si no entran, se desbordarán hacia afuera de la pantalla.
- **wrap**: El autobús es inteligente. Si un pasajero ya no entra físicamente a lo ancho, lo manda elegantemente a la "siguiente fila" debajo.

Ejemplo Práctico de un Menú de Navegación Perfecto:

```
.navbar {  
  display: flex; /* Enciende el motor de Flexbox */
```

```
flex-direction: row; /* El láser apunta a la derecha */  
justify-content: space-between; /* Lanza el logo a la izquierda y los  
enlaces a la derecha */  
align-items: center; /* Centra todo perfectamente a nivel vertical */  
}
```

3. El Espaciado Moderno: gap (La bendición de CSS)

Olvida los horribles cálculos de márgenes negativos. `gap` es una propiedad moderna que pones en el Padre, y mágicamente crea un espacio uniforme solo **ENTRE** los hijos, pero nunca en los bordes.

Traducción Técnica: `gap` (también llamado `gutter`) es un valor que el motor de renderizado inyecta entre los rectángulos de colisión de los hijos. A diferencia del `margin`, no se colapsa ni afecta la posición del contenedor respecto a sus propios vecinos.

```
.caja-botones {  
  display: flex;  
  gap: 20px; /* Crea separación idéntica de 20px entre todos los botones  
*/  
}
```



TIP

Diferencia clave: El `margin` pertenece al hijo (empuja hacia afuera). El `gap` pertenece al padre (gestiona el vacío entre sus hijos). ¡Usa siempre `gap` en Flexbox!

Tema 2: Contenedores e ítems modernos sin dependencia de grillas complejas.

1. Personalidad de los Pasajeros (Items Hijos)

A veces quieres que un hijo sea más codicioso que los demás. En la caja hija usamos el atajo trino `flex: [grow] [shrink] [basis];`.

- **Flex Basis (El tamaño ideal de arranque):** Funciona como un `width`, pero inteligente. Es el espacio que el elemento "pide por favor" antes de que el padre reparta el resto. Al rotar a columna, se encarga del alto automáticamente.
- **Flex Grow (Codicia de Crecimiento):** Si hay asientos vacíos en el autobús, ¿quién se acuesta en ellos? Si el Hijo-A tiene `flex-grow: 1` y el Hijo-B tiene `flex-grow: 2`, el B engordará absorbiendo el **doblo de espacio sobrante** que el A.
- **Flex Shrink (El nivel de delgadez):** Si el autobús reduce su tamaño porque estás viéndolo en celular, ¿quién se achica más rápido? Si le pones a una foto `flex-shrink: 0`, **nunca jamás** se achicará, manteniendo su calidad intocable.

| El Atajo Profesional (`flex: X`)

En una entrevista laboral, esperan que uses el atajo directo agrupado:

```
/* "Compañeros, ocupen matemáticamente el mismo espacio exacto todos" */
.item-igualitario {
    flex: 1; /* Equivale a decirle: Crece a tope, pero tomando 0% de base
inicial */
}

/* "Nunca me achiquen aunque la pantalla sea minúscula" */
.item-intocable {
    flex: 0 0 300px; /* No crece(0), No se encoge(0), Mide de base(300px)
*/
}
```

Rompiendo Reglas Individuales: **align-self** y **order**

A veces un hijo quiere rebelarse de la regla colectiva del autobús:

- **align-self:** Permite a un solo hijo desobedecer el **align-items** del padre y centrarse distinto.
- **order:** Permite reordenar visualmente las cajas con números enteros sin tener que modificar la posición de las etiquetas en tu archivo **HTML**. Vital para el modo responsive en celulares.

Documentación Oficial (Referencias)

- CSS: <https://developer.mozilla.org/es/docs/Web/CSS>
- Flexbox: https://developer.mozilla.org/es/docs/Web/CSS/CSS_flexible_box_layout/Basic_concepts_of_flexbox
- HTML: <https://developer.mozilla.org/es/docs/Web/HTML>